Reinforcement Learning: MDPs, Bellman, and SARSA

BSc Engineering — 3-hour lecture

Carlos Cotrini

2026-05-15

Block 0 — Welcome

Reinforcement Learning in 3 hours

By the end of today you will be able to:

- Read the **SARSA** algorithm
- Explain every line of the pseudocode
- Run SARSA by hand on a 4-state MDP
- Say what SARSA converges to and *why*

Roadmap

Block 1 — Markov Decision Processes · **45 min**

☕ break · 10 min

Block 2 — Returns, value functions, Bellman equations · **50 min**

☕ break · 10 min

Block 3 — ε-greedy, SARSA, Q-learning teaser · **50 min**

Block 4 — Wrap-up · **10 min**

Block 1 — Markov Decision Processes

Hook: a warehouse delivery robot

- 4×4 grid warehouse
- Pick up the **package** at one cell
- Deliver to the **depot** at another
- The floor is **slippery** — the intended move only happens 80 % of the time
- Each step **costs battery**; running out ends the episode

. . .

The question. What is the best plan? And how could the robot *learn* one without being told the rules?

What makes sequential decisions under uncertainty hard?

1. Decisions affect **future states**, not just the immediate outcome.

2. Outcomes are **random**.

3. Reward is **delayed** — the depot is many steps away.

We need a formalism that captures all three. Enter the **Markov Decision Process**.

The Markov property

Intuition. The current state is a *sufficient statistic* of the past — given S_t, knowing $S_{t-1}, S_{t-2}, \ldots$ adds nothing.

Formally:

$$\begin{aligned}\Pr\big[\, S_{t+1} = s' \,\big|\, S_0, A_0, \ldots, S_t = s, A_t = a \,\big] \\ = \Pr\big[\, S_{t+1} = s' \,\big|\, S_t = s, A_t = a \,\big]\end{aligned}$$

Sticky point

A restriction on the **state**, not on the process. If you need the past, fold it into the state.

Notation we will use today

Random variables, realizations

Symbol	Meaning
$S_t,\ A_t$	random state and action at step t
R_{t+1}	random reward after A_t in S_t
$s,\ a,\ r$	realizations / dummy variables

Value functions, hyperparameters

Symbol	Meaning
$v_\pi,\ q_\pi$	**true** value functions under π
$V,\ Q$	**estimates** (what algorithms update)
π	policy
$\gamma,\ \alpha,\ \varepsilon$	discount, step size, exploration

Sticky point

Reward is indexed $t+1$, not t. Standard Sutton & Barto convention.

An MDP is a 5-tuple $\langle \mathcal{S}, \mathcal{A}, P, R, \gamma \rangle$

Symbol	Name	Type
$\mathcal{S}$	state space	a set
$\mathcal{A}$	action space	a set
P	transition kernel	$\mathcal{S} \times \mathcal{A} \to \Delta(\mathcal{S})$
R	reward function	$\mathcal{S} \times \mathcal{A} \times \mathcal{S} \to \mathbb{R}$
γ	discount factor	scalar in $[0, 1]$

We will unpack each on its own slide, illustrated on the warehouse robot.

$\mathcal{S}$ — the state space

Everything the agent needs to know to act.

Warehouse robot, partial list:

- Position on grid: $(x, y) \in \{0, 1, 2, 3\}^2$
- Holding the package? $\in \{0, 1\}$
- Battery level: $\in \{0, 1, \ldots, B_{\max}\}$

Total: $|\mathcal{S}| = 16 \times 2 \times (B_{\max} + 1)$.

The state is whatever makes the **Markov property** hold for *your* problem.

$\mathcal{A}$ — the action space

What the agent can choose at each step.

Warehouse robot:

$$\mathcal{A} = \{\,\mathrm{N},\ \mathrm{S},\ \mathrm{E},\ \mathrm{W},\ \text{pick-up},\ \text{drop}\,\}$$

Action availability can depend on the state — `pick-up` only makes sense on the package cell. We model this either by giving an action-set $\mathcal{A}(s)$ per state, or by allowing the action everywhere and giving illegal cases a large negative reward.

P — the transition kernel

$$P(s' \mid s, a) \;=\; \Pr\!\left[\, S_{t+1} = s' \,\middle|\, S_t = s,\; A_t = a \,\right]$$

A *probability distribution* over next states. Sums to 1 over s'.

Warehouse step (slippery floor):

Outcome	Probability
Move in intended direction	0.80
Slip 90° left	0.10
Slip 90° right	0.10

A wall in the chosen direction → the robot stays in place.

> ⚠ **Warning**

R — the reward function

$$R(s, a, s') \;=\; \text{scalar reward observed when } s \xrightarrow{a} s'$$

Warehouse rewards:

- $+10$ for delivering the package to the depot
- -1 per step (battery cost)
- -100 if the battery dies before delivery

> **Tip**
>
> The reward signal communicates **what** you want — not **how** you want it achieved. *Sutton & Barto §3.2*

γ — the discount factor

$$\gamma \in [0,\ 1]$$

Future rewards are *discounted* by γ per step.

Engineering intuitions for $\gamma < 1$:

- A charged battery **now** is worth more than the same battery five steps from now.
- A sensor reading **now** is worth more than the same reading next loop iteration.
- It keeps infinite-horizon sums **finite**.

> ⚠ **Pre-empt**
>
> A *large* γ does **not** mean a short-term focus. Quite the opposite — γ close to 1 makes the agent **patient**.

Trajectories and episodes

A **trajectory** is the sequence the MDP generates:

$$S_0,\ A_0,\ R_1,\ S_1,\ A_1,\ R_2,\ S_2,\ A_2,\ R_3,\ \ldots$$

An **episode** is a trajectory that ends in a *terminal* state.

Warehouse: an episode ends when the package is delivered, or the battery dies.

Policies: deterministic and stochastic

A **policy** π tells the agent how to act.

Stochastic (the general case):

$$\pi(a \mid s) \;=\; \Pr[\, A_t = a \,\big|\, S_t = s \,]$$

Deterministic:

$$\pi(s) \;=\; a$$

Warehouse examples. Deterministic: *"always go N until on the package row, then E."* Stochastic: *"60 % N, 40 % E."*

We will see soon why we sometimes *want* stochastic policies — to **explore**.

Exercise 1 — identify the MDP components

Pair work — 3 min

A battery-powered shuttle moves along a one-way 3-stop bus route. At each stop it can `wait` (passengers may board) or `depart`. Each passenger carried gives reward $+5$; each minute waiting costs -1; arriving at the depot ends the episode.

1. Write down $\mathcal{S}$ and $\mathcal{A}$.
2. Sketch one valid trajectory of length 4.
3. Write one deterministic policy.

Exercise 1 — solution

$\mathcal{S}$: $(\text{stop} \in \{0, 1, 2\},\ \text{passengers} \in \mathbb{N}_0,\ \text{minute} \in \mathbb{N}_0)$ – the state must contain enough to decide.

$\mathcal{A}$: $\{\text{wait},\ \text{depart}\}$.

Trajectory *(one example)*:

$$(0,0,0) \xrightarrow{\text{wait}}_{R=-1} (0,1,1) \xrightarrow{\text{depart}}_{R=+5} (1,0,2) \xrightarrow{\text{wait}}_{R=-1} (1,1,3)$$

Deterministic policy: *"depart whenever passenger count* ≥ 1*; otherwise wait."*

> ⚠ **Warning**
>
> **Common error.** Confusing the **trajectory** (what happened) with the **policy** (the decision rule that produced it).

Break (10 min)

Block 2 — Returns, value functions, Bellman

The return G_t

The **undiscounted** return:

$$G_t \;=\; R_{t+1} + R_{t+2} + R_{t+3} + \dots$$

Two problems:

- May be **infinite** if the episode never ends.
- Treats reward today and reward in 1 000 steps the same.

The **discounted** return:

$$G_t \;=\; R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots \;=\; \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$$

Why expectations?

The return G_t is **random**:

- Stochastic environment (P)
- Stochastic policy (π)

Two trajectories from the same starting state can give wildly different G_t.
We need a *single number* per state to compare policies.

Solution. Take the expected value:

$$\text{value} \;:=\; \mathbb{E}_\pi[\, G_t \mid \text{starting situation} \,]$$

The subscript π means: "expectation over trajectories generated by following policy π."

State-value $v_\pi(s)$

The expected return *if you start in s and follow π forever after*:

$$\boxed{v_\pi(s) \;:=\; \mathbb{E}_\pi[\, G_t \mid S_t = s \,]}$$

Warehouse: "Value of being in this cell, given my current policy."

> **Sticky**
>
> Lowercase v_π = the **true** function (a property of the MDP and π). Uppercase V = our **estimate** (what algorithms compute).

Action-value $q_\pi(s, a)$

The expected return *if you start in s, take action a, and follow π forever after*:

$$\boxed{q_\pi(s, a) \;:=\; \mathbb{E}_\pi[\, G_t \mid S_t = s,\; A_t = a \,]}$$

The first action is **overridden**; everything after that follows π.

> **Important**
>
> q_π is the **protagonist** of the rest of the lecture. SARSA estimates $Q \approx q_\pi$.

v_π and q_π are connected

The two are linked through the policy:

$$\boxed{v_\pi(s) \;=\; \sum_a \pi(a \mid s)\, q_\pi(s, a)}$$

In words. The value of a state under π is the policy-weighted average of its action-values at that state.

A consequence: if you have q_π, you have v_π for free.

Bellman expectation: derivation, step 1

Start from the recursive form of the return:

$$G_t \;=\; R_{t+1} + \gamma\, G_{t+1}$$

Plug into the definition of v_π:

$$v_\pi(s) \;=\; \mathbb{E}_\pi\big[\, R_{t+1} + \gamma\, G_{t+1} \;\big|\; S_t = s \,\big]$$

Linearity of expectation:

$$v_\pi(s) \;=\; \mathbb{E}_\pi[R_{t+1} \mid S_t = s] \;+\; \gamma\, \mathbb{E}_\pi[G_{t+1} \mid S_t = s]$$

Bellman expectation: derivation, step 2

Condition on the action and the next state. By the **Markov property**, $\mathbb{E}_\pi[G_{t+1} \mid S_{t+1} = s']$ is just $v_\pi(s')$:

$$\mathbb{E}_\pi[G_{t+1} \mid S_t = s] \;=\; \sum_a \pi(a \mid s) \sum_{s'} P(s' \mid s, a)\, v_\pi(s')$$

Same conditioning for the immediate reward:

$$\mathbb{E}_\pi[R_{t+1} \mid S_t = s] \;=\; \sum_a \pi(a \mid s) \sum_{s'} P(s' \mid s, a)\, R(s, a, s')$$

Combine — and we get the equation on the next slide.

Bellman expectation equation for v_π ★

$$\boxed{v_\pi(s) \;=\; \sum_{a} \pi(a \mid s) \sum_{s'} P(s' \mid s, a) \Big[\, R(s, a, s') + \gamma\, v_\pi(s') \,\Big]}$$

Reading. Take an action under π → observe (r, s') → receive r, then the value of where you land — **averaged over everything random**.

A **self-consistency** equation: v_π appears on both sides.

Bellman expectation equation for q_π ★★

$$q_\pi(s,a) \;=\; \sum_{s'} P(s' \mid s,a)\Big[\, R(s,a,s') + \gamma \textstyle\sum_{a'} \pi(a' \mid s')\, q_\pi(s',a') \,\Big]$$

The inner sum equals $v_\pi(s')$, so equivalently:

$$q_\pi(s,a) \;=\; \sum_{s'} P(s' \mid s,a)\Big[\, R(s,a,s') + \gamma\, v_\pi(s') \,\Big]$$

> **THIS is the equation SARSA samples**
>
> The inner structure $r + \gamma\, q_\pi(s',a')$ for the next (s',a') is **exactly** the (S, A, R, S', A') tuple SARSA collects.

Worked example: compute q_π by hand

A 3-state chain. Actions: `left`, `right`. Transitions deterministic. Step cost -1; entering the terminal T gives reward $+10$. Discount $\gamma = 0.9$. Policy π = "always `right`."

Bellman expectation gives:

$$q_\pi(s_2, \text{right}) = +10$$
$$q_\pi(s_1, \text{right}) = -1 + \gamma\, q_\pi(s_2, \text{right})$$
$$q_\pi(s_2, \text{left}) = -1 + \gamma\, q_\pi(s_1, \text{right})$$
$$q_\pi(s_1, \text{left}) = -1 + \gamma\, q_\pi(s_1, \text{right})$$

Substituting back:

(s, a)	q_π
(s_2, right)	$+10$
(s_1, right)	$+8$
(s_2, left)	$+6.2$
(s_1, left)	$+6.2$

This **only worked** because we knew P and R.

Bellman optimality — the dream

The **optimal** value functions:

$$v_*(s) \;=\; \max_\pi v_\pi(s), \qquad q_*(s,a) \;=\; \max_\pi q_\pi(s,a)$$

There is always a deterministic optimal policy π_*. Its action-value satisfies the **Bellman optimality equation:**

$$q_*(s,a) \;=\; \sum_{s'} P(s' \mid s,a)\Big[\, R(s,a,s') + \gamma \max_{a'} q_*(s',a') \,\Big]$$

Only difference vs. Bellman expectation: $\sum_{a'} \pi(a'|s') \to \max_{a'}$. We won't dwell on q_*; the rest of the lecture **estimates q_π for a gradually-improving policy**.

The catch: in practice, P and R are unknown

To solve the Bellman equations directly, we need P and R explicitly. **Real engineering problems** rarely give us either — slip probabilities, contact dynamics, transition kernels are all unknown or approximate.

Three families of solutions:

1. **Estimate P, R from data, then solve** *(Model-based RL — out of scope.)*
2. **Average lots of full episode returns** *(Monte Carlo — high variance, episodic only.)*
3. **Sample one transition and bootstrap** *(Temporal Difference — today.)* ★

Where TD sits — sample × bootstrap

	No sampling	**Sampling**
No bootstrapping	exhaustive search	**Monte Carlo** (full returns)
Bootstrapping	**Dynamic Programming**	**Temporal Difference ★**

- **DP.** Computes the expectation exactly — needs P and R.
- **MC.** Samples whole episodes — no bootstrapping, but high variance.
- **TD.** Samples one transition, bootstraps off the current estimate. **SARSA lives here.**

★ Greedy improvement over q_π is model-free

To act greedily, we need $\arg\max_a$.

With $V(s)$:

$$\arg\max_a \sum_{s'} P(s'|s,a)\big[\, R + \gamma V(s') \,\big]$$

Needs P, R. **Model-based.**

With $Q(s,a)$:

$$\arg\max_a Q(s,a)$$

Just look up the table. **Model-free.**

> **Important**
>
> This is the reason SARSA estimates q_π instead of v_π.

Exercise 2 — write a Bellman equation

Pair work — 3 min

The warehouse robot is in cell $(2, 1)$, holding the package, and considers action N.

The slip model is the standard $80/10/10$. Going N:

- Intended (80 %): land in $(2, 2)$
- Slip-left (10 %): land in $(1, 1)$
- Slip-right (10 %): land in $(3, 1)$

Each move costs -1. The policy π is *"always go N until at the depot."*

Write the Bellman expectation equation for $q_\pi\big((2, 1, \text{has-pkg}, \text{batt} = 7), \text{N}\big)$.

Exercise 2 — solution

Apply the boxed Bellman equation. With π deterministic ($\pi(s') = \mathrm{N}$ for every landing state) the inner sum collapses to a single term, and the step cost -1 factors out:

$$q_\pi(s, \mathrm{N}) \;=\; -1 + \gamma\Big[\,0.8\, q_\pi(s_N, \mathrm{N}) \;+\; 0.1\, q_\pi(s_W, \mathrm{N}) \;+\; 0.1\, q_\pi(s_E, \mathrm{N})\,\Big]$$

> ⚠ **Watch for**
>
> *(1)* Dropping the $\pi(a'|s')$ weighting (matters once π is stochastic). *(2)* Double-counting the reward inside the inner sum. *(3)* Taking $\max_{a'}$ — that's Q-learning, not Bellman expectation for q_π.

Break (10 min)

Block 3 — ε-greedy and SARSA

Driving home — a TD intuition pump

You're driving home from work. At each landmark you keep an estimate of total time to home.

- **Office (start).** Estimate: 30 min.
- **Landmark 2.** No new info → still ~30 min.
- **Landmark 3.** Hit unexpected traffic. Realise you're going to be late.

Question. Do you wait until you actually get home to update your **office** estimate? Or do you bump it **right now**, using your updated estimate at landmark 3?

> **Important**
>
> Of course you bump it now. That is **bootstrapping**. This is the heart of TD — and you don't need an MDP to feel it.

SARSA combines two ideas

To turn the Bellman expectation equation for q_π:

$$q_\pi(s,a) \;=\; \sum_{s'} P(s'|s,a)\Big[R(s,a,s') + \gamma \sum_{a'} \pi(a'|s')\, q_\pi(s',a')\Big]$$

into something we can **learn from experience**, we make two substitutions.

Idea 1 — Sampling. Replace the expectations $\sum_{s'} P,\ \ \sum_{a'} \pi$ by single observed samples.

Idea 2 — Bootstrapping. Replace the unknown $q_\pi(s', a')$ by our current **estimate** $Q(s', a')$.

The next two slides go deep on each idea.

Idea 1 — Sampling

Bellman has $\sum_{s'} P(s'|s,a) \cdot [\dots]$ — an expectation we cannot compute (we don't know P).

But the environment **hands us a sample** s' every time we take action a in state s.

Trick. Replace the expectation by **one** observed transition:

$$\mathbb{E}[X] \;\approx\; X^{\,(\text{observed})}$$

Same idea applies to the inner $\sum_{a'} \pi(a'|s')$: just take **one** action a' ourselves.

Idea 2 — Bootstrapping

The Bellman expression contains $q_\pi(s', a')$ — a quantity we **don't know**.

But we **have** an estimate $Q(s', a')$. Use it.

> **Tip**
>
> **TD updates a guess towards a guess.** — *David Silver, UCL Lecture 4*

Yes, the target is a guess. Yes, it's biased. But it gets **better** as Q improves — the algorithm bootstraps itself.

The TD update for Q

Putting both ideas together:

$$Q(s,a) \leftarrow Q(s,a) + \alpha \Big[\underbrace{r + \gamma\, Q(s',a')}_{\text{TD target}} - \underbrace{Q(s,a)}_{\text{current estimate}} \Big]$$

The bracketed quantity is the **TD error**:

$$\delta = r + \gamma\, Q(s',a') - Q(s,a)$$

Reading. Nudge the estimate $Q(s,a)$ a fraction α of the way toward what Bellman would say *if we trusted our current estimates and one observed transition*.

Why we need exploration — the trap

Acting **greedily** with $Q \equiv 0$ initially. Ties broken by action order $\mathrm{N} \prec \mathrm{S} \prec \mathrm{E} \prec \mathrm{W}$.

Step 1. All Qs tied at 0 → robot picks N → step cost -1 → $Q(\mathrm{start}, \mathrm{N}) = -0.5$.

Step 2. $Q(\mathrm{start}, \mathrm{N}) = -0.5$ is now worst. Greedy picks the next-tied action S → after the update it too goes negative.

> ⚠ **Warning**
>
> The robot **never tries** E or `pick-up` long enough to discover reward. Q for those stays at 0, so they "look good" forever — but never get a real chance.

Why we need exploration — the fix

The fix is to **occasionally pick a non-greedy action**.

That is the role of ε-greedy on the next slide.

ε-greedy

A simple, near-universal exploration rule.

$$A_t = \begin{cases} \arg\max_a Q(S_t, a) & \text{with probability } 1 - \varepsilon \\ \text{uniform random action} & \text{with probability } \varepsilon \end{cases}$$

Behaviour at the extremes:

ε	**behaviour**
0	pure exploit — the catastrophe on the previous slide
1	pure random walk — never uses what it has learned
0.1	typical default: 90 % greedy, 10 % “try something new”

Tip

ε schedules

Constant ε (e.g. 0.1). Simple. Works in many problems. But the policy stays *forever* stochastic, so it can never reach the optimum.

Decaying ε (e.g. $\varepsilon_t = 1/t$). Explore early; exploit late.

GLIE

Greedy in the Limit with Infinite Exploration. A schedule that: - visits every (s, a) infinitely often, **and** - becomes greedy in the limit.

This is the formal condition that lets SARSA converge to the optimal policy.

SARSA backup diagram

The five elements of one update step, drawn as a chain:

$$\underbrace{S}_{\text{state}} \longrightarrow \underbrace{A}_{\text{action}} \longrightarrow \underbrace{R}_{\text{reward}} \longrightarrow \underbrace{S'}_{\text{next state}} \longrightarrow \underbrace{A'}_{\text{next action}}$$

- A chosen ε-greedily from $Q(S, \cdot)$
- R, S' observed from the environment
- A' chosen ε-greedily from $Q(S', \cdot)$

The chain **is** the algorithm.

Why "SARSA"?

The five elements of the chain:

$$\left(\mathbf{S},\ \ \mathbf{A},\ \ \mathbf{R},\ \ \mathbf{S}',\ \ \mathbf{A}' \right)$$

> **Tip**
>
> > *"This quintuple gives rise to the name **Sarsa** for the algorithm." — Sutton & Barto, §6.4*

The name IS the mnemonic.

SARSA pseudocode — the whole algorithm

```
Initialize Q(s, a) arbitrarily for all s, a; set Q(terminal, ·) = 0
Choose hyperparameters α ∈ (0, 1], γ ∈ [0, 1], ε ∈ [0, 1]

Repeat (for each episode):
    S ← initial state of the episode
    A ← ε-greedy(Q, S)

    Repeat (for each step of episode):
        Take action A; observe R, S'
        A' ← ε-greedy(Q, S')
        Q(S, A) ← Q(S, A) + α [ R + γ Q(S', A') - Q(S, A) ]
        S ← S'
        A ← A'
    until S is terminal
```

> **Important**
>
> Every symbol on this slide has appeared on a previous slide. We have arrived.

Visualising Q on the gridworld

Each cell is **quartered** — one triangle per action $\{N, S, E, W\}$. The number in the triangle is $Q(s, a)$. The arrow leaving the cell shows $\arg\max_a Q(s, a)$.

What you should see develop over training:

- Cells near the depot acquire the highest values first.
- Value "flows backwards" from the depot through the gridworld.
- Arrows align with the optimal-or-near-optimal direction of travel.

Hand-trace SARSA — setup and first 2 steps

Setup. Robot at $(0, 0)$, no package, full battery. $Q \equiv 0$. $\alpha = 0.5$, $\gamma = 0.9$, $\varepsilon = 0.1$.

Step 1. $A = \mathrm{N}$ (greedy, tie-break order). Bumps wall: $S' = (0, 0)$, $R = -1$. $A' = \mathrm{S}$ (next tie).

$$Q((0,0),\mathrm{N}) \;\leftarrow\; 0 + 0.5\,[-1 + 0.9 \cdot 0 - 0] \;=\; -0.5$$

Step 2. $A = \mathrm{S}$. Lands at $(0, 1)$, $R = -1$. $A' = \mathrm{N}$.

$$Q((0,0),\mathrm{S}) \;\leftarrow\; 0 + 0.5\,[-1 + 0.9 \cdot 0 - 0] \;=\; -0.5$$

Hand-trace SARSA — step 3 and the insight

Step 3. Robot at $(0,1)$. $A = \mathrm{N}$ (all Qs here still 0). Lands at $(0,0)$. $R = -1$. At $(0,0)$ now, E is the highest action (N, S are at -0.5). $A' = \mathrm{E}$.

$$Q((0,1),\mathrm{N}) \;\leftarrow\; 0 + 0.5\,[-1 + 0.9 \cdot 0 - 0] \;=\; -0.5$$

> **Note**
>
> With $Q \equiv 0$ and step cost -1, **every** step makes the chosen action look worse — so greedy naturally keeps switching, even without ε. SARSA is *learning that nothing has worked yet*.

Convergence — the headline result

Sutton & Barto, §6.4

> SARSA converges with probability 1 to an optimal policy and action-value function as long as **all state–action pairs are visited an infinite number of times** and the policy **converges in the limit to the greedy policy** (which can be arranged, for example, with ε-greedy policies by setting $\varepsilon = 1/t$).

Two conditions, no proof today:

1. **GLIE.** Visit every (s, a) infinitely often; become greedy in the limit.
2. **Robbins–Monro on α.** $\sum_t \alpha_t = \infty$ and $\sum_t \alpha_t^2 < \infty$.

"With probability 1" = almost-sure convergence, in the measure-theoretic sense — same notion you saw in your stochastic processes course.

Showcase — Windy Gridworld

A 7×10 grid. **Wind** blows the agent upward by 1 or 2 cells in the middle columns, regardless of the chosen action. Start S, goal G. Step cost -1.

SARSA with $\alpha = 0.5, \varepsilon = 0.1, \gamma = 1$:

- Finds a working path in ~ 100 episodes.
- Stable solution: ~ 17 steps per episode.
- Optimal (with this wind, no ε): 15 steps. The extra 2 are the cost of exploration.

Teaser: Q-learning swaps one symbol

SARSA:

$$Q(S,A) \leftarrow Q(S,A) + \alpha \Big[R + \gamma \, \mathbf{Q(S', A')} - Q(S,A) \Big]$$

Target uses the action A' that the agent will actually take.

Q-learning:

$$Q(S,A) \leftarrow Q(S,A) + \alpha \Big[R + \gamma \max_{\mathbf{a}'} \mathbf{Q(S', a')} - Q(S,A) \Big]$$

*Target uses the **best** possible next action — regardless of what the agent will do.*

> **Important**
>
> One symbol changed. Profoundly different behaviour. Why? Next slide.

SARSA vs Q-learning — the Cliff

A grid with a **cliff** along the bottom row. Stepping into the cliff: $R = -100$ and reset to start. Every other step: $R = -1$.

- **SARSA** learns the **safe** path — one row above the cliff. It *accounts for* the ε-greedy slip that occasionally pushes it off.
- **Q-learning** learns the **optimal-but-risky** path — along the cliff edge. It pretends it will play greedily forever.

> **The punchline**
>
> **SARSA** learns about the policy it's *actually following*.
>
> **Q-learning** learns about the policy it *wishes* it could follow.

Exercise 3 — one SARSA update by hand

Pair work — 3 min

Hyperparameters. $\alpha = 0.1, \gamma = 0.9$.

Q-table (only what you need):

$$Q(s_2, \text{right}) = 5.0 \qquad Q(s_3, \text{left}) = 2.0$$

One observed transition:

$$S = s_2,\ A = \text{right},\ R = -1,\ S' = s_3,\ A' = \text{left}$$

Compute the new $Q(s_2, \text{right})$.

Exercise 3 — solution

Plug into the SARSA update and simplify:

$$
\begin{aligned}
Q(s_2, \text{right}) \leftarrow\ & Q(s_2, \text{right}) + \alpha \left[R + \gamma\, Q(s_3, \text{left}) - Q(s_2, \text{right}) \right] \\
=\ & 5.0 + 0.1 \cdot \left[-1 + 0.9 \cdot 2.0 - 5.0 \right] \\
=\ & 5.0 + 0.1 \cdot (-4.2)
\end{aligned}
$$

$$
\boxed{Q(s_2, \text{right}) \ = \ 4.58}
$$

⚠ Common errors

(1) Using $\max_{a'}$ — that's Q-learning. *(2)* Treating $Q(S, A)$ on the RHS as the just-updated value. *(3)* Reading $\gamma\, Q(s', a')$ as $\gamma + Q(s', a')$.

Block 4 — Wrap

From MDP to SARSA in one diagram

$$\underbrace{\langle \mathcal{S}, \mathcal{A}, P, R, \gamma \rangle}_{\text{MDP}} \xrightarrow{\text{policy value}} q_\pi \xrightarrow{P,R \text{ unknown}} \underbrace{\text{sample} + \text{bootstrap}}_{\text{TD update}}$$

$$\xrightarrow{\text{must explore}} \underbrace{\text{TD update} + \varepsilon\text{-greedy}}_{} = \boxed{\text{SARSA}}$$

> **Tip**
>
> We **defined** the problem (MDP) → **characterised** what we want (q_π via Bellman) → **bypassed** the unknown model (sample + bootstrap) → **fixed** exploration (ε-greedy) → **arrived** at SARSA.

What we did *not* cover

Named, not taught — for the curious:

Other tabular methods

- Q-learning (beyond the teaser)
- Expected SARSA, n-step SARSA
- SARSA(λ), eligibility traces

Beyond tabular RL

- Function approximation, Deep RL
- Policy gradients
- Continuous actions, off-policy

Where to read next: Sutton & Barto (free PDF) · David Silver UCL · Stanford CS234.

> **Tip**
>
> You now have the language to read and understand any of these.